Characterizing Human-Agent Dynamics in Agent-authored Pull Requests

Anonymous Author(s)

***Abstract*—Pull requests (PRs) are becoming the primary interface through which coding agents contribute to software repositories, yet how these contributions are actually reviewed remains poorly understood. Existing evaluations largely reduce agent-authored PRs to a single endpoint, i.e., merged or not, hence collapsing fundamentally different review histories into the same label: a PR integrated without human attention looks identical to one that underwent several rounds of substantive revision. We argue that agent-authored PRs must instead be studied as workflows, and reconstruct four stages, Entry, Feedback, Feedback-Response Cycles, and Integration, from public GitHub timelines across a same-window corpus of 325,523 agent-authored PRs, with human and bot PRs as context, linked to comments, reviews, commits, actor metadata, and outcomes. Our analysis shows that most agent-authored PRs never enter substantive review at all, and when they do, the reviewer is nearly as likely to be another agent as a human, meaning first-line triage of agent-generated code is itself increasingly automated. Decomposing PRs into workflow pathways further reveals that the vast majority follow one of two shallow trajectories (no feedback or feedback with no code response), while the minority that undergo iterative revision behave in systematically different, more consequential ways. Interestingly, the PRs with the cleanest-looking review outcomes are often those with the least actual iteration, suggesting that visible approval signals can mask an absence of scrutiny rather than confirm its presence. These findings show that merge rate and review presence are poor proxies for how agent-authored contributions are actually vetted and argue for treating PR-thread workflow signals as first-class evaluation criteria for coding agents.**

TBD-01: Update the abstract after the content analysis; distinguish visible feedback from actual scrutiny.

***Index Terms*—AI Agents, Pull Requests, Code Review, HCI**

I. Introduction

TBD-02: Retain the workflow motivation; introduce the content of interaction as the remaining question.

Coding agents are moving from local completion and chat assistance toward repository-facing tools that can plan tasks, edit multiple files, run checks, and open pull requests (PRs) on GitHub [1]–[3]. This shift relocates where agent work becomes accountable because a patch produced by an agent is not project state until it passes through the repository's PR workflow. The PR thread is therefore the gate at which human developers and an agent's output actually meet.

Yet we still evaluate that output almost entirely by its endpoint.

TBD-03: Revise this broad claim to acknowledge existing studies of review loops and validation.

Coding-agent benchmarks and recent studies of agent-authored PRs report task completion, benchmark accuracy, code change, testing, security, reviewer engagement, review loops, acceptance, and merge rates [4]–[14]. These measures matter, but a merge label collapses everything happening between PR creation and integration. The same "merged" outcome can describe a PR that received no human attention and was integrated silently, one that made a short exchange followed by a fix, or several rounds of revision. These cases are identical as outcomes, but not as process.

Opening the thread, however, is not straightforward. PRs are socio-technical workflows shaped by technical risk, contributor history, project norms, and reviewer workload [15]–[17], and agent-authored PRs add new measurement hazards. Feedback may originate from humans, agents, or automation bots, and raw PR-thread messages may mix review-relevant feedback with deployment, status, helper, or check boilerplate. Characterizing how agent PRs are handled therefore requires reconstructing the PR timeline as layered evidence rather than merely counting whether a PR was reviewed or merged.

We take this view and study agent-authored PRs as feedback-response-revision-integration workflows. From public PR timelines, we distinguish raw visible feedback from retained review-relevant feedback, identify who first engages, summarize feedback body form and intent, reconstruct feedback-response cycles, and relate the resulting workflow pathways to integration endpoints. The contribution is not a new endpoint predictor or a new isolated PR feature family. It is a PR-level reconstruction of how entry, feedback, actor, post-feedback code activity, and endpoint compose in the same public workflow.

We assemble a large-scale, same-window corpus of 325,523 agent-authored PRs from 2025-04-01 to 2026-01-31, together with 29,808 human- and 1,206 bot-authored PRs as same-window context, and link each PR to its comments, reviews, commits, actor metadata, and outcome. Four research questions (RQ) drive our analysis.

TBD-04: Keep four RQs; embed requirements, grounds, and explicit handling within RQ2 to RQ4.

- **RQ1 (Entry).** How often do agent-authored PRs enter PR-thread interaction, and who enters first?
- **RQ2 (Feedback).** Where is feedback delivered, who provides it, and what body forms and intents appear?
- **RQ3 (Feedback-response cycles).** What workflow pathways, cycle latencies, and responses follow feedback?
- **RQ4 (Integration).** How do entry, feedback, and response cycles compose with integration outcomes?

Our results show that agent-authored PRs follow no single review path, and that the endpoint hides most of it. Only

26.6% of marked agent PRs receive any feedback, while retained review-relevant feedback remains a minority across strict, main, and relaxed filtering boundaries (10.3–13.3%, with 10.5% under the main definition), and 10.3% contain at least one feedback-response cycle. Among retained-feedback PRs, the first responder is nearly evenly split between humans (50.0%) and agents (47.4%), with automation bots accounting for 2.5%. In other words, close to half of retained-feedback first-line triage of agent work is itself performed by other agents. The derived pathways make the heterogeneity concrete because most agent PRs have no feedback (73.4%) or feedback with no code response (13.6%), while other pathways are smaller but behaviorally distinct. Post-feedback revision is sparse overall yet concentrated in a heavy tail of cycle-bearing PRs. Tellingly, late-feedback PRs, i.e., those whose final thread event is approval-like or merge-adjacent, show the highest merge rate, meaning the cleanest "successful review" outcomes often involve the least iteration.

These findings argue that coding-agent evaluation should report PR-thread workflow signals, including feedback entry, retained feedback, first-entry actor, feedback body form, feedback intent, response timing, post-feedback revision, and integration endpoint, instead of reducing them to pass, review, or merge labels. Our contributions are as follows.

TBD-05: Explain which additional workflow distinctions content associations establish; reserve unvalidated mechanisms for Discussion.

1) **In-the-wild PR workflow corpus.** We construct a large-scale, same-window dataset linking agent-authored PRs, human PRs, PR-thread comments, feedback bodies, actor metadata, and integration outcomes. The accompanying repository provides the analysis scripts, generated tables and figures, source maps for reported numbers, and a 50% random sample of the dataset for verification.[1]
2) **Human-agent PR workflow framework.** We characterize agent-authored PRs as an Entry, Feedback/Response Cycle, and Integration process, with measures for triager, feedback actors, body form, cycle timing, response, and ending, plus intent labels for aggregate feedback summaries.
3) **Empirical characterization of agent-authored PR workflows.** We characterize how agent-authored PRs enter feedback, who first triages them, how feedback channels, actors, body forms, and intents differ, how much code response follows, and how workflow pathways relate to integration outcomes.

II. Methodology

A. Data Collection & Structure

TBD-06: Add annotation coverage, failed or unfinished records, and the snapshot used for content results.

The dataset contains agent-, human-, and bot-authored GitHub PRs created between 2025-04-01 and 2026-01-31. PR actions and linked metadata were collected through the GitHub GraphQL API. Agent-authored PRs are identified using the public account, branch, and text signals listed in Table I, as in prior work [6], [8]. When multiple signals match, account- and branch-based signals take precedence over text matches. For the baseline PRs, human and bot labels are assigned from the PR author's account.[2] As in prior work using account-name patterns to identify bots [18], the bot baseline includes PRs whose authors' logins match bot-name patterns, including `[bot]`, Dependabot, and Renovate. The remaining baseline PRs are labeled human-authored. Baseline filtering and deduplication are described in Section II-C.

Feedback participants use the same three labels: Human, Agent, and Bot. Their labels are determined from the accounts that post comments or reviews. Accounts matching coding-agent or review-agent patterns are labeled *Agent*. Accounts matching explicit bot markers or service-automation patterns are labeled *Automation bot*, covering dependency updates, continuous integration, coverage, code quality, deployment, contributor license agreements, and merge queues. Accounts with no agent or automation-bot signal after refinement are labeled *Human*. These labels describe account signals rather than GitHub account types or whether a service uses AI internally. Figures abbreviate *Automation bot* as *Bot*.

Each PR is identified by its repository and PR number and linked to its recorded actions. Table I summarizes the collected records before baseline filtering. PR counts use unique repository–PR keys, repository counts use unique target (base) repositories, and commit, comment, and review counts use unique GitHub event identifiers. Agent rows can overlap because a PR may match signals for multiple agents. The total row counts each record once. Table II reports the PR groups used in the analyses after filtering and deduplication.

Public user profiles provide background on the accounts participating in agent-authored PRs. The existing profile table contains 58,262 accounts with account-creation dates and public contribution summaries. PR-thread accounts are linked to these profiles by GitHub login. Accounts without a matching profile remain in the PR analyses but are excluded from analyses requiring profile data. Profile measures and their coverage for human responders are described in Section II-E1.

Repository metadata describe the settings in which agent-authored PRs are submitted. Each PR is linked to its target (base) repository by repository name, with names associated with multiple repository IDs left unmatched. The 325,523 agent-authored PRs span 103,883 target repositories, of which 103,865 have matching metadata. Figure 1 shows repository age at PR creation, recorded stars, the number of agent-authored PRs per repository in the study sample, and recorded primary language. Ownership, stars, and primary language describe recorded snapshots rather than historical values at PR creation. Attribute coverage is 99.92% for age, 32.51% for stars, 96.67% for primary language, and 99.89% for ownership. Among PRs with ownership metadata, 80.3% target

[1] anonymous.4open.science/r/icse27_submission-0F26

[2] PR author refers to the account recorded in the GitHub GraphQL `PullRequest.author` field: https://docs.github.com/en/graphql/reference/pulls#pullrequest.

TABLE I
REPOSITORY-VISIBLE SIGNALS AND UNIQUE COLLECTED DATASET SIZE BEFORE SCOPED BASELINE FILTERING.

Author	Repository-visible signal	Unique PRs	Base repos	Commits	Comments	Reviews
Google Jules	author:google-labs-jules[bot]	64,547	17,203	67,468	29,025	7,582
Cursor	head:cursor/	54,039	20,373	76,233	28,882	5,644
Devin	author:devin-ai-integration[bot]	49,305	7,092	99,741	56,576	13,029
OpenAI Codex	head:codex/	48,471	18,070	47,021	6,812	3,997
Claude Code	"Co-Authored-By: Claude"; "Generated with Claude Code"	46,772	14,832	131,415	35,941	20,674
GitHub Copilot	head:copilot/	44,886	20,153	108,167	42,226	31,672
JetBrains Junie	author:jetbrains-junie,junie-eap[bot]	10,472	9,596	17,018	1,345	270
Codegen	"About Codegen"	8,651	2,356	17,343	17,088	3,025
Cosine	head:cosine/	4,373	904	12,348	1,521	685
Amazon Q Developer	author:amazon-q-developer[bot]	2,129	684	2,534	7,308	10,385
OpenHands	head:openhands; {"Co-authored-by: ", "Automatic fix generated by"} "OpenHands"	1,797	340	6,529	3,423	1,556
Tembo	head:tembo/	1,024	197	1,629	903	144
Human	Human baseline within study window	31,443	22,538	95,424	18,559	21,401
Total		**367,868**	**129,993**	**682,324**	**249,539**	**120,042**

TABLE II
SCOPED PR ANALYSIS GROUPS BY PR AUTHOR CLASS. HERE BOT DENOTES THE BOT-AUTHORED BASELINE. RAW PR-THREAD ENTRY COUNTS PRS WHOSE FIRST POST-OPEN NON-AUTHOR PR-THREAD EVENT IS A COMMENT OR REVIEW.

Author	PRs	Merged	Raw PR-thread entry
Agent	325,523	224,269	86,504
Human	29,808	21,553	9,989
Bot	1,206	882	509

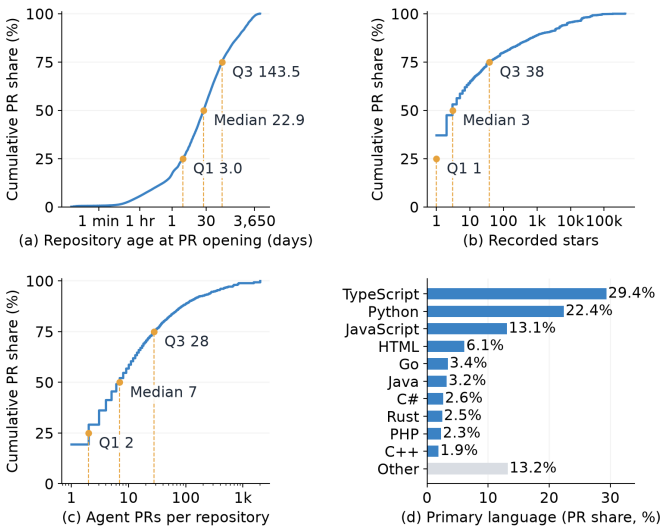


Fig. 1. Repository context of agent-authored PRs. Percentages are calculated over PRs with available values for each attribute. Panels (a)–(c) show cumulative PR shares on logarithmic x-axes. Panel (d) shows the ten most common primary languages, with the remaining 234 grouped as Other.

repositories owned by personal accounts and 19.7% target organization-owned repositories.

B. PR Timeline Construction

TBD-07: Keep the original non-author feedback definition; state the wider source scope of the new content analysis separately.

Feedback refers to recorded `Comment` or `Review` events posted by an account other than the PR author, with timestamps strictly later than the PR opening time. These events are grouped as feedback because PR comments and reviews can guide subsequent changes by coding agents [19], [20].

For each PR, the timeline combines `Open`, `Comment`, `Review`, and `Commit` records linked to that PR. Each record identifies the account, event type, and timestamp. The earliest `Open` record identifies the PR opening time and author account. Records are ordered by timestamp, with code commits recorded separately from feedback.

The first feedback event is the earliest event that meets the definition above. Comments and reviews posted by the PR author do not qualify, but the author's later code commits remain in the timeline. When a `Comment` and a `Review` have the same timestamp, the `Comment` is selected using a fixed tie-breaking rule. Measures of who provides the first feedback include only PRs with feedback.

C. Baseline construction

Human- and bot-authored PRs serve as references for feedback and integration in agentic contributions. Both groups were collected through the same GitHub GraphQL API procedure, including PR records and their associated events. They are distinguished using the author signals described in Section II-A. Each PR is linked to its events by repository and PR number, with GraphQL PR identifiers mapped to this pair where available. The available human and bot baseline records span PR creation dates from 2025-06-01 to 2025-11-14. Within this period, duplicate records with the same repository and PR number are reduced to one record per PR, keeping the earliest recorded creation time. Table II reports the resulting groups.

These comparisons describe how feedback and integration patterns vary across agent-, human-, and bot-authored PRs. Across all three author groups, PR records are linked to events by repository and PR number and deduplicated by retaining the record with the earliest creation timestamp. The baselines provide context for the observed differences.

D. Human-Agent PR Workflow Model

TBD-08: Add the content layer to the workflow explanation and figure without changing the four stages.

Recent research examines discussions among developers triggered by automated review comments [21]. This motivates examining both who participates in the discussion and what feedback they provide, captured here as participation and feedback. Research on human and LLM review comments further distinguishes comment types from whether developers resolve them [22]. This distinction motivates examining revision separately from feedback, focusing on the code changes that follow it. Finally, research on pull-based development examines merge decisions and processing time [23]. Integration connects the preceding interactions to the PR's observed status. These perspectives motivate four research questions that characterize RQ1 Participation, RQ2 Feedback, RQ3 Revision, and RQ4 Integration within the same PR timeline (Figure 2).

RQ1 examines which PRs receive feedback and who provides the first feedback. For PRs with feedback, RQ2 examines its channels, forms, and content. RQ3 then examines code changes after feedback, including their timing, authors, and size. RQ4 relates these interaction patterns to whether PRs are merged, closed without merging, or still open at data collection.

E. Taxonomy Construction

1) RQ1: Participation: **Feedback and Participants.** RQ1 examines how many PRs receive feedback and who provides the first feedback. The share of PRs receiving feedback is calculated before and after removing empty messages and specified status or boilerplate messages, such as deployment notices and CI summaries. Section II-F details these filtering rules. Both calculations use all PRs in each author group as the denominator and count a PR if it has at least one feedback event. After filtering, the earliest remaining feedback event identifies the first feedback provider for each PR.

The first feedback provider is classified as Human, Agent, or Automation bot using the signals in Section II-A. The automated rules left 15 accounts unclassified. Manual inspection assigned these accounts to Human (10), Agent (1), and Automation bot (4). Only the first feedback event remaining after applying the message-filtering rules in Section II-F is used, so each PR contributes to only one category. Category shares use all PRs with at least one remaining feedback event within each author group as the denominator.

Human Participant Profiles. For agent-authored PRs whose first feedback after message filtering comes from a human, the provider is matched to the public user profiles introduced in Section II-A. Matching uses GitHub usernames converted to lowercase with surrounding whitespace removed. Matched profiles provide account-creation dates and public contribution counts for the account measures defined below. PRs without the required profile data remain in the feedback analysis but are excluded from comparisons based on these measures.

Public GitHub Footprint Score. The public GitHub footprint score combines account age and public contribution counts. For each user u, $\mathrm{tenure}(u)$ is the number of days from account creation to the latest user-profile collection timestamp. Public contribution volume is defined as $\mathrm{contrib}(u) = \#\mathrm{commits}(u) + \#\mathrm{PRs}(u) + \#\mathrm{reviews}(u) + \#\mathrm{issues}(u)$, using the one-year window recorded in the user-profile dataset. Both components are transformed using $\log(1+x)$ and capped at their 5th and 95th percentiles. The transformed components are then min–max normalized to $[0,1]$ across all eligible user profiles. The resulting values, $\widehat{\mathrm{tenure}}(u)$ and $\widehat{\mathrm{contrib}}(u)$, receive equal weights:

$$\mathrm{footprint}(u) = 0.5\,\widehat{\mathrm{tenure}}(u) + 0.5\,\widehat{\mathrm{contrib}}(u).$$

Footprint Groups. Score quartiles are calculated among human participants in agent-authored PRs with available scores. The 25th, 50th, and 75th percentiles define four groups, Q1 to Q4, from the lowest to the highest scores. For PRs whose first feedback comes from a human with an available score, the PRs are grouped by that person's score quartile. Figure 3 shows the score distribution and the share of these PRs in each group.

2) RQ2: Feedback: **TBD-09:** Define requirements, completion claims, reported grounds, assessments, and explicit textual associations.

RQ2 examines feedback events on agent-authored PRs after applying the message filters in Section II-F. Each feedback event is analyzed separately, so a PR can be associated with multiple feedback events. The recorded feedback type distinguishes two channels, Comment and Review. Within each channel, feedback is further grouped by provider type.

Feedback actors. Feedback providers are classified as Human, Agent, or Automation bot using the account classification described in Section II-E1. The share of each provider type is calculated across all feedback events after filtering and separately for comments and reviews. For the separate calculations, the denominator is the number of comments or reviews, respectively.

Human feedback providers are matched to the public user profiles and assigned to the footprint quartiles defined in Section II-E1. Feedback from human accounts without an available footprint score remains in the analysis and is reported separately.

Text Features.

TBD-10: Compress format and length summaries to make room for requirement-level content analysis.

Prior work has examined textual features of review comments [24] and identified verbosity and information presentation as concerns in bot communication [25]. Motivated by these studies, the analysis focuses on feedback length and

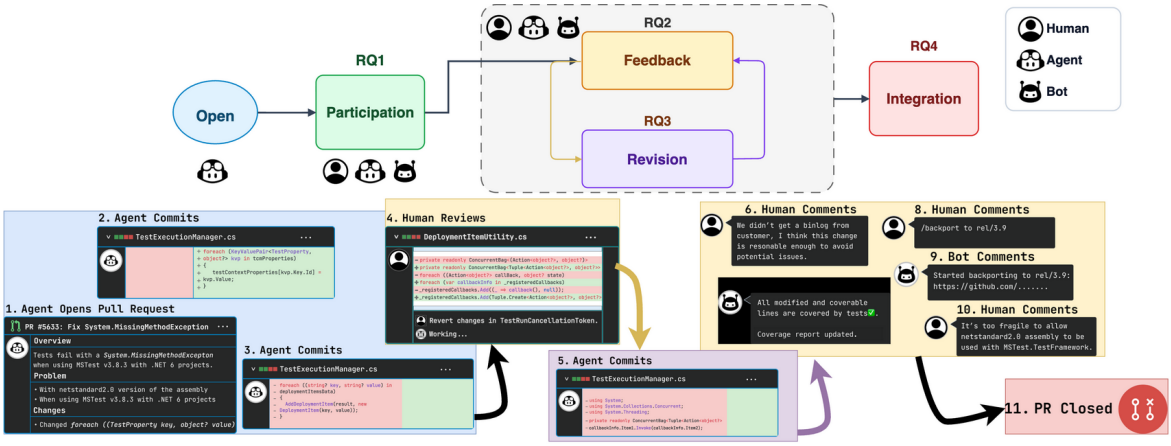


Fig. 2. Overview of the four research questions and an example PR timeline. The upper panel shows the research questions of participation, feedback, revision, and integration at their relative positions in a PR timeline, where feedback and revision may repeat. The lower panel illustrates feedback, revision, and closure in PR microsoft/testfx#5633.

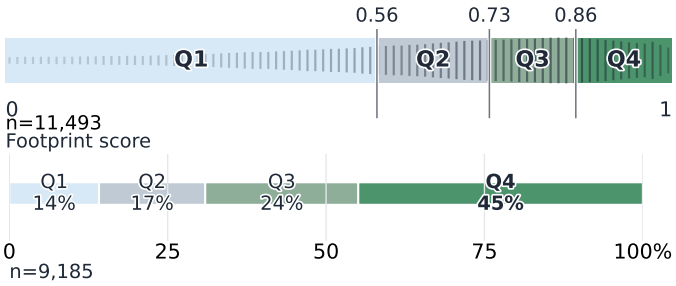


Fig. 3. Top: human participants' footprint scores and quartile boundaries. Bottom: PR shares by the first feedback provider's quartile, restricted to PRs whose first feedback after filtering comes from a human with an available score.

format. Length is measured by word and line counts for messages with available, nonempty text. Format is characterized using text-matching rules for code-block markers, tables, lists, summary-like structures, and status-related expressions. Their frequencies and the distributions of feedback length are summarized by provider type, separately for comments and reviews.

Intent Taxonomy. To describe what the feedback concerns, RQ2 groups messages by their primary intent. Prior research has used comment categories to compare human-written and LLM-generated code reviews [22]. Motivated by this work, the taxonomy distinguishes seven categories: *Functional*, *Maintainability*, *Documentation*, *Discussion*, *Process*, *Validation*, and *Non-actionable*. Each feedback event receives one primary label.

Functional concerns incorrect or missing behavior. *Maintainability* concerns code clarity and structure. *Documentation* covers requests to change documentation or explanatory text. *Process* covers build, CI, deployment, and integration requests. *Validation* covers requests for tests, reproduction steps, or other evidence. *Discussion* covers questions and clarification without a specific change request. *Non-actionable* covers approvals, acknowledgments, and other messages without a request for further action or clarification. Generated feedback is classified by its content using the same categories. For messages containing multiple requests, the primary label represents the dominant request.

Intent Labeling. Human annotation supports the development and assessment of the LLM classifier. Prior work has also assessed LLM-based review classification using a manually labeled sample of 100 comments [22]. In this study, sampling covers different PR author types, feedback providers, merge states, and feedback and revision patterns. Up to four events were selected from each stratum, producing 187 candidates. Selection within strata favored shorter messages with explicit requests and excluded status-only messages. A stratified allocation of approximately 70% of these candidates, with rounding within strata, yielded 141 events for human annotation.

Two annotators independently assigned primary intent labels to the 141 events. They then discussed disagreements and revised labels for 41 events. After this discussion, their labels agreed on 104 events, with Cohen's $\kappa = 0.683$. The remaining 37 events were unresolved and excluded from the reference set used for classifier development and evaluation. Following the existing split, the reference set contained 75 development events and 29 evaluation events.

The development set supplied examples for the classification prompt. Model and prompt configurations were compared on the 29-event evaluation subset. The selected configuration uses `gpt-4o-mini`, two examples per category, and batches of 20 feedback events. None of the evaluation events appears among the prompt examples. The model returns one primary intent label for each event. On the evaluation subset, this configuration matched the human reference labels for 20 of 29 events, yielding 0.690 accuracy and 0.676 weighted-F1. Weighted-F1 averages category-level F1 scores according to their frequencies in this subset. All outputs contained a valid

category label. The selected configuration was then applied to the remaining feedback events.

3) RQ3: Revision: **TBD-11:** Separate subsequent commit activity from explicit textual responses; treat modification claims as reports.

RQ3 examines whether code commits follow feedback and how these changes unfold within a PR. The analysis uses the PR timelines defined in Section II-B to group feedback events and subsequent commits into cycles. These cycles are constructed before applying the message-content filters used in RQ2.

Blocks and cycles. Consecutive feedback events with no intervening commit form a *feedback block*. The commits that follow this block, before the next feedback event, form its *response block*. A feedback block and a response block containing at least one commit form one *feedback-response cycle*. Multiple commits within the same response block count as one cycle. A feedback block with no subsequent commit does not form a cycle.

Only commits recorded after PR opening and before any recorded merge or closure are included. Commits whose messages match predefined merge, squash, or rebase patterns are also excluded to avoid counting these operations as revisions following feedback. The remaining commits and feedback events are ordered by timestamp. At identical timestamps, comments precede reviews, and both precede commits. Events with the same timestamp and type are ordered by their identifiers to ensure a consistent ordering. A commit with the same timestamp as the preceding feedback might hence form a cycle with zero waiting time.

PR pathways. Prior work describes code review as an iterative process in which reviewer comments are followed by revisions [26]. To characterize how this process unfolds in PR timelines, the pathway classification distinguishes whether feedback occurs and whether it is followed by one or multiple revision cycles. Cycle counts alone do not show whether further feedback appears after the last revision. The classification therefore also distinguishes PRs whose final feedback block follows the last included commit. These distinctions define five pathways, with each PR assigned to exactly one using the rules below.

No visible feedback. The PR contains no feedback events. *Feedback without code response*. Feedback is present, but no cycle is formed and the PR does not meet the late-feedback rule below. *Single-feedback response cycle*. The PR contains exactly one cycle and does not meet the late-feedback rule. *Multi-feedback response cycle*. The PR contains two or more cycles and does not meet the late-feedback rule. *Late feedback after final code change*. The final feedback block follows the last included commit, with no subsequent commit.

The late-feedback rule takes precedence over cycle count. PRs in this category may contain zero, one, or multiple earlier cycles.

Cycle timing. Cycle latency is the elapsed time from the last feedback event in a feedback block to the first commit in its response block. Each cycle contributes one observation, so a PR with multiple cycles contributes multiple observations. Feedback blocks without a subsequent commit are excluded from latency summaries. The distribution is summarized using the median, interquartile range, and upper percentiles. A separate summary includes only cycles whose feedback blocks contain at least one event retained by the message filters in Section II-F. This subset uses the original cycle boundaries and latency values.

Revision size. Prior work measures PR size using commit counts, changed lines, and changed files [23]. In this study, these measures can also describe revisions within each cycle and across each PR. For each cycle, commits are counted and changed lines and file counts are summed across its response block. Changed lines are calculated as additions plus deletions when both values are available. File counts come from the per-commit `changed_files` field, so a file modified in multiple commits is counted more than once.

For each cycle, total changed lines are also divided by the number of feedback events to describe change volume relative to feedback volume. For each PR, included commits are divided into those before and after its first feedback event. PRs without feedback or subsequent commits remain in the summaries with zero commits and changes after feedback.

Revision accounts. For each response block, GitHub accounts are extracted from the author fields of its commits. The committer field is used when no author account is available.

4) RQ4: Integration: RQ4 examines how the PR pathways defined in Section II-E3 relate to observed PR states. These states are determined from the PR metadata in the dataset snapshot used for analysis, using GitHub's merge and close timestamps.[3] A PR is classified as *merged* if a GitHub merge timestamp is present, including when a close timestamp is also recorded. Otherwise, a recorded close timestamp identifies a PR as *closed without merge*. PRs with neither timestamp are classified as *open*, consistent with their recorded state.

The pathway comparison includes all agent-authored PRs in the study sample, selected by creation time regardless of their observed state. PRs without visible feedback are included in the corresponding pathway defined in Section II-E3. Within each pathway, the proportions of merged, closed-without-merge, and open PRs use all PRs in that pathway as the denominator. Merge-rate differences are reported in percentage points by subtracting the merge rate of the no-visible-feedback pathway from that of each other pathway.

Additional comparisons group agent-authored PRs by the provider, primary intent, or text form of their first feedback event remaining after message filtering. Each comparison includes PRs with the corresponding information available. The footprint comparison further restricts the sample to PRs whose first retained feedback comes from a human with an available score, using the quartiles defined in Section II-E1. For each comparison, the reference proportion is calculated across its eligible PRs before grouping. Differences in merge

[3]GitHub GraphQL API, `PullRequest.mergedAt` and `PullRequest.closedAt`: https://docs.github.com/en/graphql/reference/pulls#pullrequest.

and open proportions are reported in percentage points relative to this reference.

Time from first feedback to merge or closure is calculated separately from the PR timelines used in RQ3. For each measure, the first feedback timestamp is subtracted from the corresponding GitHub timestamp, and the duration is expressed in days. Only PRs whose first feedback strictly precedes that timestamp contribute to the timing summary. Time to closure includes merged PRs with a recorded close timestamp.

For merged PRs in the late-feedback pathway, the last feedback timestamp is compared with the merge timestamp to determine whether feedback occurs before, at the same recorded time as, or after merging. PRs in the late-feedback pathway are also grouped by whether their final feedback block contains comments, reviews, or both. For blocks containing reviews, merge proportions are further summarized by the review states recorded by GitHub, including blocks in which all reviews indicate approval.[4]

F. Validation and Robustness

TBD-12: Add validation of quotes, IDs, and content links; specify what the existing human references can validate.

Retained-feedback sensitivity. We evaluate four feedback boundaries. *Raw visible feedback* includes all post-open non-author PR comments and reviews. *Relaxed retained feedback* removes only empty-body events and comments or reviews from known status, deployment, CI, helper, routing, release, security-scan, and administrative service accounts. *Main retained feedback* further excludes deployment previews, CI/check/build summaries, helper invitations, routing or assignment commands, generated summary shells without review findings, release/version notices, CLA/administrative messages, security-scan notices, and other boilerplate/status-only events matched by the retained-feedback codebook. We use this main retained boundary for retained-entry, feedback-actor, body-form, and intent summaries. Structural pathway analyses keep the broader visible-feedback timeline and state the denominator where results appear. The filter is content/status based, and agent or bot messages are retained if they contain substantive review, process, evidence, or change-request content. To sanity-check, the authors jointly audited a stratified sample of 100 retained and 100 excluded feedback events and found no systematic rule violations. *Strict retained feedback* additionally removes short non-actionable acknowledgments and terminal signals, such as approval-only, thanks-only, LGTM-only, no-comments/no-issues, and merge-ready comments, unless they contain actionable cues such as fix, test, validate, refactor, or failure-related requests. Across non-raw retained definitions, retained-feedback entry remains a minority of agent-authored PRs. Relaxed, main, and strict retained boundaries cover 13.3%, 10.5%, and 10.3% of agent-authored PRs, respectively.

[4]GitHub records approval as `APPROVED` in `PullRequestReview.state`: https://docs.github.com/en/graphql/reference/pulls#pullrequestreviewstate.

Footprint-score sensitivity. We recompute footprint strata using tenure-only, contribution-only, and alternative tenure/contribution weights, while keeping each PR's first-human-responder assignment fixed. PRs whose first human responder lacks scoreable metadata remain outside these footprint-stratified summaries.

Endpoint censoring sensitivity. Because open or without-close endpoints may reflect the collection boundary, we repeat integration summaries after excluding PRs opened near the end of the study window and after exclusions that leave one top agent out, so that endpoint patterns are not affected by late-window censoring or a single dominant agent. Across the main and sensitivity summaries, proportions are reported with counts and denominators, with Wilson or difference intervals for selected actor, syntax, and endpoint contrasts in the artifact.

Repository characteristics.

TBD-13: Retain existing repository analyses; add context checks for the new content measures and report comparison coverage.

To examine how workflow patterns vary across repositories, agent-authored PRs are grouped by repository ownership, repository age at PR creation, and the number of sampled PRs from each repository. Within each group, the analysis calculates the proportions of PRs with visible feedback, at least one feedback-response cycle, and a recorded merge.

Ownership distinguishes repositories owned by personal accounts from those owned by organizations. Repository age is divided into four descriptive groups: up to 90 days, over 90 to 365 days, over 365 to 1,095 days, and over 1,095 days. The number of sampled agent-authored PRs per repository is grouped as 1, 2–10, 11–100, and more than 100.

To show variation across repositories, the three proportions are also calculated separately for each repository within each group. Their distributions are displayed as empirical cumulative distribution functions. A repository can appear in more than one age group because its PRs may have been created at different repository ages.

Group differences are estimated using linear probability models with group membership as the predictor. The models use standard errors clustered by repository to account for dependence among PRs from the same repository [27]. Differences are reported in percentage points with 95% confidence intervals.

For each repository characteristic, an F-test assesses whether the groups have equal proportions for each of the three measures. Holm correction is applied across the nine tests and separately across all group-to-reference comparisons [28]. Groups with unknown characteristics are excluded from these tests.

III. Results

A. RQ1: Entry

TBD-14: Retain participation coverage and first actors; shorten profile detail unless it explains a content result.

Entry denominators. Among 325,523 agent-authored PRs, 86,504 (26.6%) have raw entry through a non-author comment

or review, 34,049 (10.5%) retain review-relevant feedback after filtering, and 33,502 (10.3%) contain at least one feedback-response cycle. The retained-entry denominator is therefore much narrower than the raw visible PR-thread record, but it is the relevant denominator for asking who first enters the PR thread with substantive retained feedback.

First retained entry actor. In the 34,049 retained-entry PRs, the first retained actor is nearly split between humans (17,037 PRs, 50.0%, Wilson 95% CI [49.5, 50.6]) and agents (16,132 PRs, 47.4%, [46.8, 47.9]). Automation bots account for 856 PRs (2.5%, [2.4, 2.7]) and unresolved agent/bot accounts for 24 PRs (0.1%, [0.0, 0.1]). RQ1's answer is therefore that retained PR-thread feedback occurs for a minority of agent-authored PRs. When it occurs, the first retained entry actor is almost evenly split between humans and agents.

First human triager footprint. Figure 3 stratifies the human side of RQ1 using an equal-weight account footprint score with 0.5 normalized account tenure and 0.5 normalized public contribution volume. The score-distribution plot shows the covered account distribution over this continuous score, with quartile cut points at 0.56, 0.73, and 0.86. Among the 17,037 PRs whose first retained entry came from a human, 9,185 (53.9%) join to a scoreable engaging account in the footprint table. The first-human-triager plot displays this scoreable subset. Q1 accounts provide the first retained human entry for 1,301 PRs (14.2%), Q2 accounts for 1,540 PRs (16.8%), Q3 accounts for 2,219 PRs (24.2%), and Q4 accounts for 4,125 PRs (44.9%). The remaining 7,852 first-human PRs (46.1% of the full first-human denominator) come from accounts whose metadata is unavailable and are therefore not assigned to quartiles. These no-metadata PRs correspond to 2,809 unique first-human triager accounts that appear in the retained-feedbacks but have no match in the user metadata and footprint-score table. The no-metadata stratum denotes uncovered metadata. Thus, among the 53.9% of first-human retained-entry PRs with scoreable metadata, developers in the largest footprint quartile account for 44.9% of first human entries.

B. RQ2: Feedback

TBD-15: Add requirements and grounds with separate PR and item denominators; keep unfinished correspondence results pending.

Feedback channels and syntax features. Figure 4 summarizes the channel and body-shape side of RQ2. The RQ2 denominator is 76,936 retained feedback events with deduplicated event IDs. Most retained feedback appears as PR comments, with 56,589 events (73.6%). GitHub review events account for the remaining 20,347 events (26.4%). Figure 4 shows that feedback on agent-authored PRs is mostly thread comment feedback rather than review events.

The body text also differs strongly by actor. Human feedback is usually short. Human comments have a median of 18 words and one line, while human reviews have a median of 6 words and one line. The one-line pattern for human reviews is real in the data because the 75th percentile is still one line, and the 90th percentile is five lines. Agent feedback is much longer. Agent comments have a median of 462 words and 68 lines, while agent reviews have a median of 254 words and 51 lines.

The syntax features tell the same story in another way. Human feedback rarely uses structured markup features such as code blocks, tables, bullets, or summary-like format. Agent feedback uses these features much more often, notably in review events. Compared with human reviews, agent reviews are +84.3 pp higher on summary-like form (95% CI [+83.6, +85.0]) and +39.9 pp higher on bullets ([+39.0, +40.9]); in comments, the summary-like gap is +38.6 pp ([+38.0, +39.3]). Figure 4 also shows that comments are the larger feedback channel, and agent feedback is more structured.

Feedback actors. The top row of Figure 5 answers the actor part of RQ2 across all retained feedback and by channel. On all retained feedback, agents provide 35,115 events (45.6%). Human accounts provide 40,231 events (52.3%), and automation bots provide 1,590 (2.1%). This all-event view differs from RQ1 because it asks who enters first, while RQ2 asks who provides the feedback events that appear after entry.

The actor mix changes by channel. Agents provide 11,188 of 20,347 reviews (55.0%) versus 23,927 of 56,589 comments (42.3%), a +12.7 pp difference between reviews and comments (95% CI [+11.9, +13.5]). Automation bots provide 2.8% of comments but only 0.1% of reviews. Figure 5 shows review events are more agent-heavy than comments.

The human side is not evenly distributed across footprint strata. Among humans, Q1 provides 3,682 retained feedback events (4.8% of all retained feedback events), Q2 provides 3,362 events (4.4%), Q3 provides 5,525 events (7.2%), and Q4 provides 10,977 events (14.3%). Another 16,685 events (21.7%) come from humans without footprint metadata. As in RQ1, this no-metadata group denotes accounts that are not able to join the footprint-score table. Within the human strata, Q4 accounts have the majority of human feedback.

Feedback intent. The bottom row of Figure 5 summarizes intent labels attached to feedback events. The overall distribution is concentrated in a few intent types. Non-actionable feedback accounts for 28,837 events (37.5%). Process feedback has 28,683 events (37.3%). Documentation feedback is the next largest group with 7,836 events (10.2%). The remaining intent groups are smaller than 5% of all feedback events.

The comparison between reviews and comments is more stable than the actor and body-shape comparison. Every intent category appears in both comments and reviews, and their shares are similar. For reviews, non-actionable and process feedback account for 36.0% and 38.4%. For comments, they account for 38.0% and 36.9%. The largest absolute channel differences are about two percentage points (pp). Discussion feedback is lower in reviews than in comments, Maintainability feedback is higher in reviews, and Process feedback is slightly higher in reviews. These differences are much smaller than the actor and syntax differences shown above.

Figures 4 and 5 show that retained feedback differs most by channel, actor, and body form. Intent differences between comments and reviews are smaller.

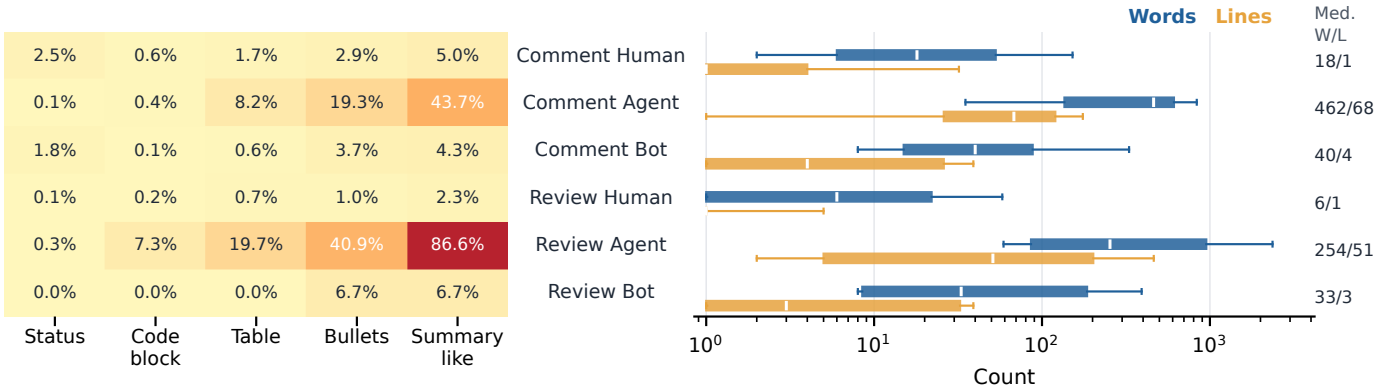


Fig. 4. RQ2 feedback body form by channel and actor. Left heatmap cells report the percentage of retained feedback events in each channel-actor group whose body matches each syntax feature. Right boxplots summarize feedback-body word and line counts on a log scale. Boxes show the IQR, whiskers P10–P90, numeric labels mark medians. Outliers hidden. Bot=automation bot.

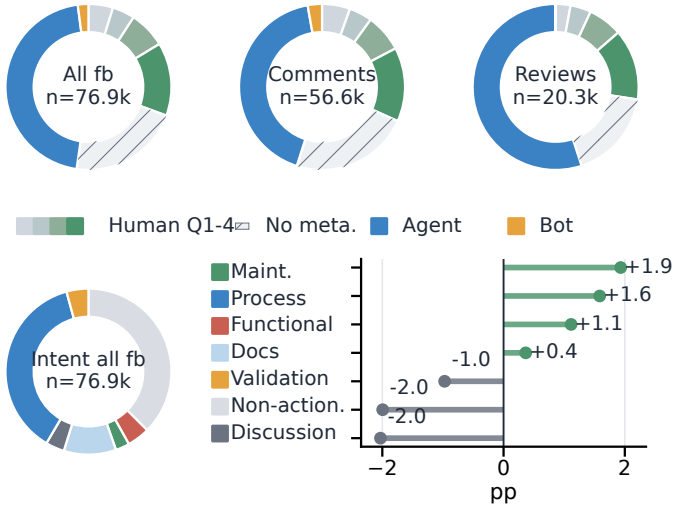


Fig. 5. RQ2 event-level feedback actors and intent mix for 76,936 retained feedback events. Panels show actor shares, primary-intent shares, and review-minus-comment intent differences in pp. Maint.=Maintainability, Docs=Documentation, Non-action.=Non-actionable, Bot=automation bot.

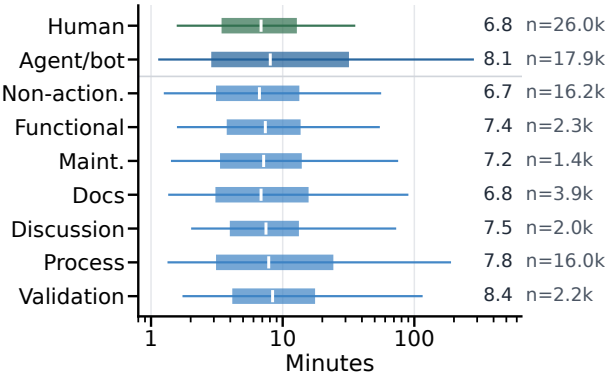


Fig. 6. Cycle-level feedback-response latency for 59,159 visible feedback-response cycles across 33,502 agent-authored PRs. Actor rows stratify cycles by feedback actor. Agent/bot combines agent accounts and automation bots. Intent rows pool cycles across actors and stratify by primary feedback intent. Thick bands show IQR, whiskers P10–P90, white ticks/numbers medians, and right labels row counts.

C. RQ3: Feedback-response Cycles

TBD-16: Add support, challenge, interpretation, clarification, and reported-action patterns alongside the temporal pathways.

Workflow pathway composition. Over all, most agent-authored PRs do not enter a feedback-response pathway. There are 239,019 PRs (73.4%) with no visible feedback. Another 44,177 PRs (13.6%) receive feedback but have no later response-eligible commit. Explicit response cycles are less common. Only 16,446 PRs (5.1%) have one feedback-response cycle, and 4,682 PRs (1.4%) have multiple cycles. The remaining 21,199 PRs (6.5%) end with late feedback after the final code change. Thus, RQ3 shows that response cycles are uncommon. Most PRs have no visible feedback or feedback without later code response.

Cycle latency. Among agent-authored PRs with feedback-response cycles, the latency table contains 59,159 cycles across 33,502 PRs. The median cycle latency is 7.0 minutes. Only 54 of 59,159 cycles (0.09%) have zero latency from same-timestamp ordering, so the 7.0-minute median is not driven by the tie-breaking rule. The majority of cycles are fast, with 62.1% occurring within 10 minutes and 88.9% occurring within 60 minutes. The distribution is however long-tailed, with P90 at 76.1 minutes and P95 at 483.2 minutes. The retained-feedback cycle subset is similar at the median. It contains 31,394 retained cycles across 16,775 PRs with a median latency of 7.1 minutes and P90 of 65.0 minutes. Because cycle latency is skewed, we emphasize medians and upper percentiles rather than averages. Figure 6 visualizes the skew with compact log-scale interval strips. Figure 6 shows the same median-fast, tail-heavy pattern by actor and intent.

Response. In the 59,159 agent-authored feedback-response cycles, response commit accounts are often not limited to the PR author. The largest response-actor category is PR author + others, covering 25,825 cycles (43.7%). Mixed non-author response accounts cover another 19,848 cycles (33.6%). Non-author human-only response blocks account for 7,955 cycles (13.4%), while PR-author-only response blocks account

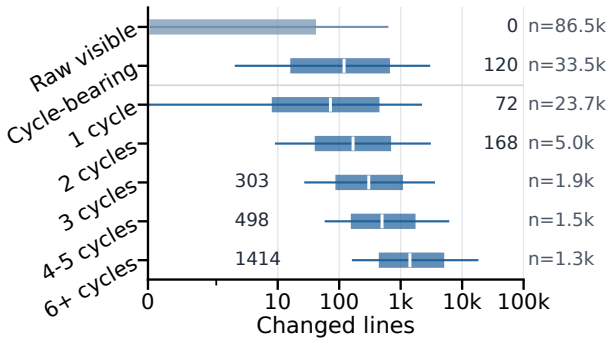


Fig. 7. Code-response magnitude after visible feedback. Bands show IQR, whiskers P10–P90, white ticks/numbers mark medians. Outliers are hidden.

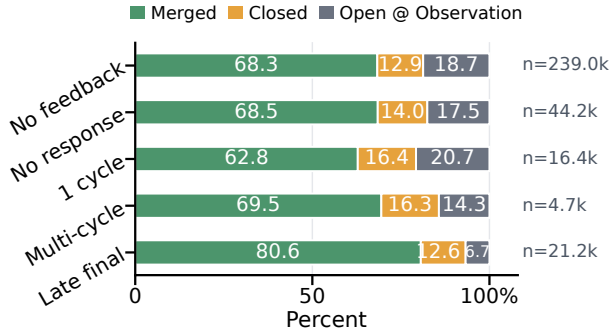


Fig. 8. PR-level RQ4 integration endpoint composition by feedback-response pathway for 325,523 agent-authored PRs. Each PR is assigned one endpoint (merged, closed without merge, or open PR at observation). Each row sums to 100%, and right labels report pathway denominators.

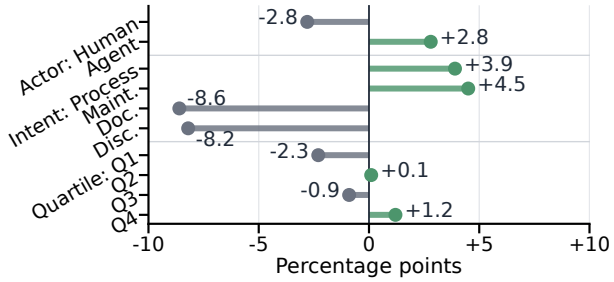


Fig. 9. PR-level RQ4 selected merged-rate differences from construct-specific baselines. Zero is the merged rate within each construct's covered subset before stratification (first retained actor, first retained intent/body form, or scoreable first-human triagers). Points are pp differences from that baseline.

for 4,290 cycles (7.3%). Non-author agent/bot-only response blocks are rather rare at 1,241 cycles (2.1%). The five strata are mutually exclusive. They are PR-author-only, PR author plus other accounts, non-author human-only, non-author agent/bot-only, and mixed non-author accounts with both human and agent/bot non-author commit accounts. These categories report commit accounts attached to GitHub commits.

Figure 7 reports the code-response magnitude after visible feedback. For PRs with raw visible feedback, changed lines remain sparse (median 0; P90 631; P95 2,074), but the feedback-with-code-response subset is larger at the median and tail (median 120; P90 3,024; P95 8,164). The cycle-count strata show that repeated cycles are associated with larger response. One-cycle PRs have a median of 72 post-feedback changed lines, whereas PRs with six or more cycles have one of 1,414 changed lines. The upper tail also grows with cycle count, from P95 of 6,010 changed lines for one-cycle PRs to P95 of 49,791 changed lines for PRs with six or more cycles. The outlier check reports changed-lines P95 of 8,164 in the visible-feedback and code-response denominator, 6,440 after dropping the top 1%, and 3,407 after dropping PRs above 10,000 changed lines. There is hence a two-level pattern. Many raw-visible-feedback PRs have no measured post-feedback code response, while cycle-bearing PRs have larger, heavy-tailed responses.

D. RQ4: Integration

TBD-17: Compare assessment content within the same pathway and outcome; verify the candidate contrasts before final reporting.

Merge outcomes over the entry-cycle composition. Figure 8 composes the RQ1 entry and RQ3 feedback-response pathway summaries with integration endpoints. The unit of analysis is the agent-authored PR, and each is assigned one endpoint category. The endpoint categories are merged, closed without merging, and open PR. The pathway-specific endpoint percentages use the same PR-level denominators introduced in RQ3. These denominators are 239,019 no-visible-feedback PRs, 44,177 feedback-without-code-response PRs, 16,446 single-cycle PRs, 4,682 multi-cycle PRs, and 21,199 late-feedback PRs. No-visible-feedback PRs are the pathway reference, with a 68.3% merge rate. Relative to that reference, feedback-without-code-response PRs are nearly unchanged (+0.1 pp, 95% CI [-0.3, +0.6]), single-cycle PRs are lower (-5.5 pp, [-6.3, -4.7]), multi-cycle PRs are similar (+1.1 pp, [-0.2, +2.5]), and late-feedback PRs are higher (+12.3 pp, [+11.7, +12.9]); single-cycle PRs also have the largest open PR share (20.7%).

Terminal feedback as an integration signal. The late-feedback pathway needs a terminal interpretation because its final feedback block starts after the final code change and has no later response block. The terminal diagnostic breakdown supports this interpretation: 11,940 late-feedback PRs end with a review-only final feedback block and merge at 89.0%; another 8,282 end with an `APPROVED`-only review state and merge at 93.8%. Among merged late-feedback PRs, 3,536 record last feedback after the merge timestamp and 9,371 within five minutes before merge. This pattern indicates that late feedback often records approval-like or merge-adjacent integration evidence. The body-form endpoint breakdown is consistent with that interpretation: summary-like first retained bodies are 10.7 pp above the body-form baseline on merged, while short and long plain-text bodies are 4.2 and 3.6 pp below it. For open PR endpoints, the pattern reverses: summary-like first retained bodies are 4.8 pp below the body-form baseline, while short and long plain-text bodies are 2.0 and 1.6 pp above it.

Integrated interpretation. Figure 9 reports merged-rate differences from construct-specific baselines for selected ac-

tor, intent, and first-human footprint-quartile strata. For first retained actor, the agent row is 2.8 pp above the actor baseline on merged, while the human row is 2.8 pp below it. For the first retained intent, Process and Maintainability rows are 3.9 and 4.5 pp above the intent baseline on merged, while Documentation and Discussion are 8.6 and 8.2 pp below it. For scoreable first-human triagers, Q1 and Q3 are 2.3 and 0.9 pp below the footprint baseline on merged, while Q2 and Q4 are 0.1 and 1.2 pp above it. For open PR endpoints, the breakdown behind Figure 9 shows the largest positive contrasts in documentation-first PRs and Q1 first-human triagers: +13.4 pp from the intent baseline and +7.0 pp from the footprint baseline, respectively. Summary-like first retained bodies are 4.8 pp below the body-form baseline on open PR, and Q4 first-human triagers are 3.9 pp below the footprint baseline. The Q1 row has the smallest quartile denominator (1,301 PRs). Together, Figures 8 and 9 show that integration endpoints vary with the preceding workflow pathway, so similar merge labels can hide different entry, feedback, and response-cycle states.

IV. Discussion

TBD-18: Explain what content associations reveal beyond workflow counts; keep conditional judgment as a candidate interpretation.

We study agent-authored PRs as engagement workflows, not as acceptance outcomes. The results do not collapse to one signal. Retained feedback, response cycles, revision, and integration capture different parts of the PR path. A raw ”reviewed” or ”merged” label can therefore mix PRs with no retained feedback, feedback without response, iterative cycles, terminal approval-like feedback, or larger revisions.

A. Implications for the Agent Development Lifecycle (ADLC)

TBD-19: Tie implications to observed review activity and reported grounds rather than inferred agent capability.

Existing ADLC discussions give names to evaluation, observability, and guardrails around agents [29], [30]. Our data add the repository boundary. An agent patch has not finished its lifecycle when a PR opens. It still has to engage with, or fail to engage with public feedback, actor participation, code response, and integration. Only 26.6% of agent-authored PRs receive raw visible feedback, retained review-relevant feedback remains 10.3% to 13.3% across filtering boundaries, and retained-feedback first entry is nearly split between humans and agents. For ADLC, observability should cover repository uptake as well as agent-internal execution: which generated changes receive review-relevant feedback, who provides it, and whether later PR activity produces code response.

B. Workflow Metrics for Agent-authored Software Changes

The pathway results explain why merge should not stand alone. A merged agent-authored PR may have no visible feedback, feedback without code response, one response cycle, repeated cycles, or terminal merge-adjacent feedback. These states point to different review and verification states, but they now collapse under one endpoint label. Integration should therefore be read together with the preceding pathway to see whether feedback appeared, whether a code response followed, and whether the final feedback block was terminal.

C. Threats to Validity and Future Work

TBD-20: Add annotation, visibility, edit-time, missing-link, and cross-chunk boundaries for the new analysis.

The corpus prioritizes precision and repository-visible signals. Following prior agent-authored PR mining work [6]–[8], the study identifies agent-authored PRs from public account, branch, and text signals. This improves attribution precision, but it cannot recover agent-assisted work hidden by private settings, disabled signals, or human accounts. The same issue may affect the human baseline, and non-repository-matched author groups make cross-group and endpoint contrasts descriptive rather than causal.

The workflow reconstruction captures public PR-thread traces: comments, reviews, commits, endpoints, and timestamps. It omits private review, local agent iteration, IDE/chat traces, and agent-runtime state. Feedback-response cycles and post-feedback revision are temporal signals; fuller traces would be needed to establish causality. Body-form and terminal-review endpoint patterns may also reflect repository automation and review conventions.

The retained-feedback filter and public GitHub footprint score also limit the constructs. Retained feedback is a conservative review-relevant subset, not a complete measure of all useful feedback. Footprint strata summarize public trace profiles rather than expertise or quality, and no-metadata accounts remain uncovered. Two endpoint robustness checks are stable: pathway endpoint rates are unchanged under the 30-day mature-PR filter, and late-feedback merge rates remain 79.1–81.1% in top-agent leave-one-out checks. Null-hypothesis significance testing is kept out of the main claims, while interval and effect-size outputs remain in the artifact. The findings are most directly informative for public GitHub repositories during 2025–2026 where agent activity is visibly marked and PR-based development is routine. Future work can connect these workflows to downstream maintenance signals and richer IDE, chat, CI, and agent-runtime traces.

V. Related Work

TBD-21: Update the closest-work comparison and move the concrete distinction into the Introduction.

a) Coding Agents in Software Development: AI support for programming has moved from local completion and chat asssitance toward tools that can plan tasks, edit multiple files, run checks, and interact with repository infrastructure [2], [3], [31]. Benchmark studies still emphasize task completion, benchmark accuracy, or issue resolution [4], [5]. These studies measure coding capability more than repository uptake after an agent-produced change becomes a PR, while recent work shows that human feedback remains important during agent-assisted planning and code generation [32]. This study examines the later repository-facing stage after PR creation.

b) Pull Requests as Socio-Technical Workflows: Pull requests are socio-technical workflows, not only containers for code changes, and are shaped by technical risk, contributor reputation, project norms, and reviewer workload [15], [16]. Prior work links PR latency to contributor familiarity, project load, and PR characteristics [17], and shows that review and acceptance outcomes depend on technical and social factors rather than code features alone [33]. Recent work also studies PRs as coordination structures, including overdue-PR interventions and links between PRs and issues [34], [35]. Mining studies warn that large-scale GitHub analyses inherit missing actions, noisy account labels, and selection bias [36]; this motivates the construct limits in Section IV-C.

c) Review Automation and Human-Agent Feedback: Modern PR threads contain human review, automation-bot messages, and increasingly agent-mediated review. Code-review bot research shows that automation can alter pull-request dynamics, developer workflows, and notification burden [25], [37], [38]. Large-scale evidence from review conversations shows that human reviewers provide broader feedback than agents, including understanding, testing, and knowledge transfer, and that adoption differs for AI and human suggestions [39]. This work motivates the actor layer, which separates humans, agents, and automation bots before analyzing response cycles, revision, and integration.

d) Empirical Studies of Agent-Authored Pull Requests: Recent empirical studies have made agent-authored PRs a rapidly growing target [1], [19]. Some characterize acceptance, merge outcomes, activity patterns, code change over time, and broad attributes [6], [8], [12]. Others focus on code modifications and PR-description consistency [9], test inclusion [10], security-related PRs [11], failed or rejected PRs [40], and cross-PR references [41].

The closest work examines review and collaboration signals around agent-authored PRs, including PR descriptions, human responses, reviewer engagement, actionable loops, and minimal-feedback merges [7], [13], [14]. This paper complements that cluster by reconstructing raw and retained feedback entry, first-entry actor, feedback actor, and intent, response-cycle latency, post-feedback revision, and pathway-conditioned integration endpoints.

VI. Conclusions

TBD-22: Refresh the conclusion from verified results; remove interpretations that exceed the visible public record.

We reframe agent-authored PRs as engagement workflows rather than binary outcomes. Only 26.6% of 325,523 agent-authored PRs receive any visible feedback, and just 10.5% carry feedback substantive enough to retain under our main filter. Where retained feedback appears, triage is nearly split between humans and agents, meaning close to half of the first-line review of agent work is itself performed by agents, a finding that merge rate alone cannot surface. The pathway decomposition shows why this matters. No-feedback and feedback-without-response PRs together account for 87.0% of the corpus, yet the remaining pathways diverge sharply. A "merge" label conflates PRs that were rubber-stamped, silently integrated, or genuinely revised, states that matter for trust and oversight but are invisible at the endpoint. By making entry, retained feedback, response cycles, and integration separately measurable, our framework offers the ADLC community a way to ask not just whether agent contributions are merged but also how.

REFERENCES

[1] H. Li, H. Zhang, and A. E. Hassan, "The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering," Jul. 2025, arXiv:2507.15003 [cs] version: 1. [Online]. Available: http://arxiv.org/abs/2507.15003

[2] C. Treude and M. A. Gerosa, "How Developers Interact with AI: A Taxonomy of Human-AI Collaboration in Software Engineering," Jan. 2025, arXiv:2501.08774 [cs] version: 1. [Online]. Available: http://arxiv.org/abs/2501.08774

[3] V. Chen, A. Talwalkar, R. Brennan, and G. Neubig, "Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows," Sep. 2025, arXiv:2507.08149 [cs]. [Online]. Available: http://arxiv.org/abs/2507.08149

[4] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, Dec. 2024, arXiv:2308.11432 [cs]. [Online]. Available: http://arxiv.org/abs/2308.11432

[5] S. Liang, S. Garg, and R. Z. Moghaddam, "The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason," Dec. 2025, arXiv:2506.12286 [cs]. [Online]. Available: http://arxiv.org/abs/2506.12286

[6] M. Watanabe, H. Li, Y. Kashiwa, B. Reid, H. Iida, and A. E. Hassan, "On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub," Feb. 2026, arXiv:2509.14745 [cs]. [Online]. Available: http://arxiv.org/abs/2509.14745

[7] K. Watanabe, R. Tsuchida, T. Monno, B. Huang, K. Yamasaki, Y. Fan, K. Shimari, and K. Matsumoto, "How AI Coding Agents Communicate: A Study of Pull Request Description Characteristics and Human Review Responses," Feb. 2026, arXiv:2602.17084 [cs]. [Online]. Available: http://arxiv.org/abs/2602.17084

[8] R. M. Popescu, D. Gros, A. Botocan, R. Pandita, P. Devanbu, and M. Izadi, "Investigating autonomous agent contributions in the wild: Activity patterns and code change over time," in *Proceedings of the 23rd international conference on mining software repositories (MSR)*. New York, NY, USA: ACM, 2026, accepted to MSR 2026; arXiv:2604.00917. [Online]. Available: https://arxiv.org/abs/2604.00917

[9] D. Ogenrwot and J. Businge, "How AI Coding Agents Modify Code: A Large-Scale Study of GitHub Pull Requests," Jan. 2026, arXiv:2601.17581 [cs]. [Online]. Available: http://arxiv.org/abs/2601.17581

[10] S. Haque, S. Ingale, and C. Csallner, "Do Autonomous Agents Contribute Test Code? A Study of Tests in Agentic Pull Requests," Jan. 2026, arXiv:2601.03556 [cs]. [Online]. Available: http://arxiv.org/abs/2601.03556

[11] M. L. Siddiq, X. Zhao, V. C. Lopes, B. Casey, and J. C. S. Santos, "Security in the Age of AI Teammates: An Empirical Study of Agentic Pull Requests on GitHub," Jan. 2026, arXiv:2601.00477 [cs]. [Online]. Available: http://arxiv.org/abs/2601.00477

[12] H. Yoshioka, T. Monno, H. Tokumasu, T. Wakamatsu, Y. Ota, N. Weeraddana, and K. Matsumoto, "Let's Make Every Pull Request Meaningful: An Empirical Analysis of Developer and Agentic Pull Requests," Jan. 2026, arXiv:2601.18749 [cs]. [Online]. Available: http://arxiv.org/abs/2601.18749

[13] C. Nachuma and M. Zibran, "When AI Teammates Meet Code Review: Collaboration Signals Shaping the Integration of Agent-Authored Pull Requests," Feb. 2026, arXiv:2602.19441 [cs]. [Online]. Available: http://arxiv.org/abs/2602.19441

[14] H. Gao, P. Banyongrakkul, H. Guan, M. Zahedi, and C. Treude, "On Autopilot? An Empirical Study of Human-AI Teaming and Review Practices in Open Source," Jan. 2026, arXiv:2601.13754 [cs]. [Online]. Available: http://arxiv.org/abs/2601.13754

[15] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in GitHub," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, May 2014, pp. 356–366. [Online]. Available: https://dl.acm.org/doi/10.1145/2568225.2568315

[16] O. Kononenko, T. Rose, O. Baysal, M. Godfrey, D. Theisen, and B. de Water, "Studying pull request merges: a case study of shopify's active merchant," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 124–133. [Online]. Available: https://dl.acm.org/doi/10.1145/3183519.3183542

[17] Y. Yu, H. Wang, V. Filkov, P. Devanbu, and B. Vasilescu, "Wait for It: Determinants of Pull Request Evaluation Latency on GitHub," in *2015 IEEE/ACM 12th Working Conference on Mining Software Repositories*. Piscataway, NJ, USA: IEEE, May 2015, pp. 367–371, ISSN: 2160-1860. [Online]. Available: https://doi.org/10.1109/MSR.2015.42

[18] W. Schueller, J. Wachs, V. D. P. Servedio, S. Thurner, and V. Loreto, "Evolving collaboration, dependencies, and use in the Rust open source software ecosystem," *Scientific Data*, vol. 9, no. 1, p. 703, 2022.

[19] H. Li, H. Zhang, and A. E. Hassan, "AIDev: Studying AI Coding Agents on GitHub," Feb. 2026, arXiv:2602.09185 [cs.SE]. [Online]. Available: http://arxiv.org/abs/2602.09185

[20] GitHub, "Copilot coding agent: Improved pull request review experience," Aug. 2025. [Online]. Available: https://github.blog/changelog/2025-08-05-copilot-coding-agent-improved-pull-request-review-experience/

[21] M. Schaef, B. Cirisci, L. Luo, M. N. Mansur, O. Tripp, D. Sanchez, Q. Zhou, and M. B. Zafar, "Understanding developer-analyzer interactions in code reviews," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2024, pp. 1945–1955.

[22] S. Goldman, H. Y. Lin, J. Pasuksmit, P. Thongtanunam, K. Tantithamthavorn, Z. Wang, R. Zhang, A. Behnaz, F. Jiang, M. Siers, R. Jiang, M. Buller, M. Jeong, and M. Wu, "What Types of Code Review Comments Do Developers Most Frequently Resolve?" Oct. 2025, arXiv:2510.05450 [cs]. [Online]. Available: http://arxiv.org/abs/2510.05450

[23] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," in *Proceedings of the 36th International Conference on Software Engineering*. ACM, 2014, pp. 345–355.

[24] M. M. Rahman, C. K. Roy, and R. G. Kula, "Predicting usefulness of code review comments using textual features and developer experience," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. Piscataway, NJ, USA: IEEE, May 2017, pp. 215–226. [Online]. Available: https://dl.acm.org/doi/10.1109/MSR.2017.17

[25] M. Wessel, I. Wiese, I. Steinmacher, and M. A. Gerosa, "Don't Disturb Me: Challenges of Interacting with SoftwareBots on Open Source Software Projects," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW2, pp. 1–21, Oct. 2021, arXiv:2103.13950 [cs]. [Online]. Available: http://arxiv.org/abs/2103.13950

[26] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: a case study at google," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '18. New York, NY, USA: Association for Computing Machinery, May 2018, pp. 181–190. [Online]. Available: https://dl.acm.org/doi/10.1145/3183519.3183525

[27] A. C. Cameron and D. L. Miller, "A practitioner's guide to cluster-robust inference," *Journal of Human Resources*, vol. 50, no. 2, pp. 317–372, 2015.

[28] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[29] D. Bergmann, "What is the agent development lifecycle (ADLC)?" May 2026, last modified May 11, 2026. Accessed: June 30, 2026. [Online]. Available: https://www.ibm.com/think/topics/agent-development-lifecycle-adlc

[30] A. Osmani, S. Saboo, and S. Kartakis, "The New SDLC with Vibe Coding," Jun. 2026, published June 16, 2026. Accessed: June 30, 2026. [Online]. Available: https://www.kaggle.com/whitepaper-the-new-SDLC-with-vibe-coding

[31] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," Feb. 2023, arXiv:2302.06590 [cs]. [Online]. Available: http://arxiv.org/abs/2302.06590

[32] W. Takerngsaksiri, J. Pasuksmit, P. Thongtanunam, C. Tantithamthavorn, R. Zhang, F. Jiang, J. Li, E. Cook, K. Chen, and M. Wu, "Human-In-the-Loop Software Development Agents," Jan. 2025, arXiv:2411.12924 [cs]. [Online]. Available: http://arxiv.org/abs/2411.12924

[33] T. Dey and A. Mockus, "Effect of Technical and Social Factors on Pull Request Quality for the NPM Ecosystem," in *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, ser. ESEM '20. New York, NY, USA: Association for Computing Machinery, Oct. 2020, pp. 1–11. [Online]. Available: https://dl.acm.org/doi/10.1145/3382494.3410685

[34] C. Maddila, S. S. Upadrasta, C. Bansal, N. Nagappan, G. Gousios, and A. v. Deursen, "Nudge: Accelerating Overdue Pull Requests Towards Completion," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 2, pp. 1–30, Apr. 2023, arXiv:2011.12468 [cs]. [Online]. Available: http://arxiv.org/abs/2011.12468

[35] C. R. B. de Souza, E. Ma, J. Wong, D. Yoon, and I. Beschastnikh, "Revealing Software Development Work Patterns with PR-Issue Graph Topologies," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 106:2402–106:2423, Jul. 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3660813

[36] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014. New York, NY, USA: Association for Computing Machinery, May 2014, pp. 92–101. [Online]. Available: https://dl.acm.org/doi/10.1145/2597073.2597074

[37] M. Wessel, A. Serebrenik, I. Wiese, I. Steinmacher, and M. A. Gerosa, "Effects of Adopting Code Review Bots on Pull Requests to OSS Projects," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. United States: IEEE Computer Society, Sep. 2020, pp. 1–11, ISSN: 2576-3148. [Online]. Available: https://ieeexplore.ieee.org/document/9240622

[38] ——, "Quality Gatekeepers: Investigating the Effects of Code Review Bots on Pull Request Activities," *Empirical Software Engineering*, vol. 27, no. 5, p. 108, Sep. 2022, arXiv:2103.13547 [cs]. [Online]. Available: http://arxiv.org/abs/2103.13547

[39] S. Zhong, S. Noei, Y. Zou, and B. Adams, "Human-AI Synergy in Agentic Code Review," Mar. 2026, arXiv:2603.15911 [cs]. [Online]. Available: http://arxiv.org/abs/2603.15911

[40] R. Ehsani, S. Pathak, S. Rawal, A. Mujahid, M. M. Imran, and P. Chatterjee, "Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub," Jan. 2026, arXiv:2601.15195 [cs.SE]. [Online]. Available: http://arxiv.org/abs/2601.15195

[41] I. Khemissi, M. Chouchen, D. Wang, and R. G. Kula, "Humans Integrate, Agents Fix: How Agent-Authored Pull Requests Are Referenced in Practice," Apr. 2026, arXiv:2604.04059 [cs.SE]. [Online]. Available: http://arxiv.org/abs/2604.04059